

# Build and Optimize a High-Performance News Aggregator with React

[Back](#)**Mandatory**

## Domain

**Frontend Development**

## Skills

Component-Based Development

Frontend Development

JavaScript Programming

Performance Testing

Public Relations

Build Tool Configuration

## Difficulty

**Intermediate**

## Tools

Docker

Lighthouse

Node.Js

React

Vite

Webpack

Chrome DevTools

## Industries

E-commerce

SaaS



## Submission Received

Your submission has been received and is pending review. You can still update your submission till the effective deadline.

Submitted on 8 May 2026, 09:38 pm

Deadline: **9 May 2026, 04:59 pm**

**Overview****Instructions****Resources****Submit**

## Objective

You will build a news aggregator application, first in a deliberately unoptimized state, and then systematically refactor it for high performance. You will learn to use professional-grade profiling tools like Lighthouse and Chrome DevTools to diagnose and fix common performance bottlenecks related to Core Web Vitals (LCP, INP, CLS). This task is crucial for building modern web applications where performance directly impacts user engagement and SEO ranking.

## Description

## 1. Project Background and Core Concepts

### partnr

"performance debt" over time. This project simulates that exact scenario. You will first construct a React-based news aggregator that is intentionally slow, buggy, and inefficient. Then, you will put on your performance engineer hat to methodically identify, measure, and eliminate bottlenecks. The goal is not just to make it fast but to understand *why* it was slow and *how* each specific optimization contributes to a better user experience.

This task revolves around Google's **Core Web Vitals (CWV)**, a set of metrics that measure real-world user experience for loading performance, interactivity, and visual stability. Mastering these is non-negotiable for modern frontend developers.

#### Key Concepts Explained

1. **Largest Contentful Paint (LCP)**: This metric reports the render time of the largest image or text block visible within the viewport. A good LCP score (under 2.5 seconds) tells the user that the page is loading correctly and the main content is available. In our app, the LCP element will likely be the large hero image or the main block of articles.
2. **Interaction to Next Paint (INP)**: INP is a new metric that assesses a page's overall responsiveness to user interactions. It measures the latency of all click, tap, and keyboard interactions. A low INP (under 200 milliseconds) ensures the UI feels fluid and responsive. We will focus on the interactions for filtering and sorting the news articles.
3. **Cumulative Layout Shift (CLS)**: CLS measures visual stability by quantifying how much unexpected layout shifts occur during the page's entire lifecycle. A good score (under 0.1) means the page is stable and doesn't have content that jarringly moves around as it loads. This is often caused by images without dimensions, ads, or content being dynamically injected.
4. **Performance Profiling Tools**:
  1. **Lighthouse**: An open-source, automated tool for improving the quality of web pages. You can run it against any web page, public or requiring authentication. It has audits for performance, accessibility, progressive web apps, SEO, and more. We will use it for our high-level performance scores.
  2. **Chrome DevTools (Performance Panel)**: This is your microscope. It provides a detailed, frame-by-frame breakdown of everything the browser is doing: loading resources, parsing HTML, executing JavaScript, calculating styles, painting, and more. It's essential for diagnosing complex issues like long tasks that block the main thread and hurt INP.

## 2. Phase 1: Building the Intentionally "Slow" Application

Your first task is to build the application with a specific set of anti-patterns. This is a critical step to create a baseline for your optimization work. You will use the free and

public **HackerNews API**.

## partnr

1. Display the top 500 stories from HackerNews.
2. Show each story's title, score, author ( `by` ), and a link to the original article.
3. Include a text input to filter articles by title.
4. Include a button to sort articles by score.
5. A large, unoptimized hero image at the top of the page.

### Implementation Anti-Patterns (To Be Implemented):

1. **Network Waterfall (N+1 Requests):** First, fetch the list of top story IDs from `https://hacker-news.firebaseio.com/v0/topstories.json` . Then, inside a loop, fetch the details for each story one by one using `https://hacker-news.firebaseio.com/v0/item/{id}.json` . This creates a massive network waterfall and is a primary performance killer.

*Pseudocode Example:*

```
// In a React component
useEffect(() => {
  const fetchAllStories = async () => {
    const response = await fetch('.../topstories.json');
    const storyIds = await response.json();
    const stories = [];
    // Anti-pattern: sequential fetching in a loop
    for (const id of storyIds.slice(0, 500)) {
      const storyResp = await fetch(`.../item/${id}.json`);
      const storyData = await storyResp.json();
      stories.push(storyData);
    }
    setArticles(stories);
  };
  fetchAllStories();
}, []);
```

2. **No List Virtualization:** Render the entire list of 500+ articles directly into the DOM. This will create an enormous number of DOM nodes, leading to high memory usage and slow rendering and re-rendering, severely impacting INP when filtering or sorting.
3. **Inefficient Dependency Usage:** Import the entire `lodash` library to use just one or two functions (e.g., `debounce` or `sortBy` ). Do not use cherry-picked imports.

*Code Example:*

```
import _ from 'lodash';
// ...
const sortedArticles = _.sortBy(articles, 'score');
```

#### 4. Expensive Computations in Render:

partnr

render.

- 5. Unoptimized Images:** Find a large hero image online (at least 2MB). Include it at the top of your page with a simple `<img>` tag. Do not add `width`, `height`, `srcset`, or `loading="lazy"` attributes. This will negatively impact LCP and CLS.
- 6. No Code Splitting:** Use a standard `create-react-app` or `Vite` setup without implementing any route-based or component-based code splitting. Everything should be in one large initial JavaScript bundle.

At the end of this phase, you should have a functioning but noticeably slow application. Commit this version to a separate Git branch called `slow-version`.

### 3. Phase 2: Establishing a Performance Baseline

Before you write a single line of optimization code, you must measure. Without a baseline, you cannot prove that your changes had a positive impact.

#### How to Measure:

- 1. Run Lighthouse:** Open Chrome DevTools, go to the Lighthouse tab, select "Performance", and generate a report for your running application. Save a screenshot or a JSON export of this report. Pay close attention to the LCP, TBT (Total Blocking Time, which correlates with INP), and CLS scores.
- 2. Use Chrome DevTools Performance Panel:**
  1. Go to the "Performance" tab.
  2. Click the record button (or `ctrl+E`).
  3. Perform an action, like typing in the filter input or clicking the sort button.
  4. Stop the recording.
  5. Analyze the flame chart. Look for long tasks (indicated with a red triangle), which are periods where the main thread was blocked for over 50ms. These are your primary targets for improving INP.

#### Documenting Your Findings

Create a file named `PERFORMANCE.md` in the root of your project. In this file, create a markdown table to document your baseline and your plan. This document will be your single source of truth for this project.

#### Baseline Performance Report Table ( `PERFORMANCE.md` )

| Metric / Issue | Baseline Score / Observation | Root Cause Analysis                                     | Proposed Solution Hypothesis  |
|----------------|------------------------------|---------------------------------------------------------|-------------------------------|
| LCP            | (e.g., 8.5s)                 | Large, unoptimized hero image blocking the main thread. | Compress image, serve in WebP |

| Metric /              | Baseline Score /                                    |                                                                 | Proposed Solution                                                             |
|-----------------------|-----------------------------------------------------|-----------------------------------------------------------------|-------------------------------------------------------------------------------|
| partnr                |                                                     |                                                                 |                                                                               |
|                       |                                                     |                                                                 | and lazy load.                                                                |
| INP (from TBT)        | (e.g., TBT: 1200ms) and observed lag on input.      | Re-rendering 500+ DOM nodes on every keystroke in the filter.   | Implement list virtualization to only render visible items.                   |
| CLS                   | (e.g., 0.45)                                        | Hero image loading without dimensions, pushing content down.    | Add explicit width and height attributes to the <code>&lt;img&gt;</code> tag. |
| Bundle Size (main.js) | (e.g., 1.5MB)                                       | Importing full <code>lodash</code> library; no code splitting.  | Use cherry-picked imports for <code>lodash</code> ; implement code splitting. |
| Network Waterfall     | (e.g., 501 serial requests observed in Network tab) | Sequential <code>fetch</code> calls in a <code>for</code> loop. | Parallelize data fetching with <code>Promise.all</code> .                     |

4. Phase 3: Systematic Optimization

Now, implement your proposed solutions one by one. This is crucial: **make one change, then measure again**. This isolates the impact of each optimization. Update your `PERFORMANCE.md` file after each step with the new measurements to track your progress.

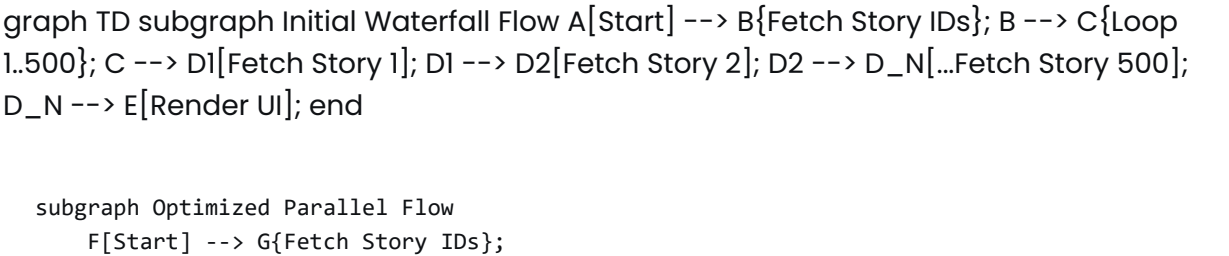
Optimization Steps:

1. Parallelize Network Requests:

- 1. **Why:** Instead of waiting for each of the 500 detail requests to finish sequentially, you can fire them all off in parallel. This dramatically reduces the total data-fetching time.
- 2. **How:** Refactor your data fetching logic to use `Promise.all`. Fetch the list of IDs first, then map over the array of IDs to create an array of promises, and pass that array to `Promise.all`.

Data Flow Architecture Diagram:

Expand



```
G --> H{Map IDs to Promises};
```

# partnr

**2. Implement List Virtualization:** \* **Why:** Rendering hundreds or thousands of elements is computationally expensive. Virtualization renders only the items currently visible in the viewport (plus a small buffer), keeping the DOM small and interactions fast. \* **How:** Use a library like `@tanstack/react-virtual`. You will provide it with the total number of items, the size of each item, and a render callback. The library then gives you the necessary styles and items to render.

## 3. Optimize Dependencies and Expensive Calculations:

1. **Why:** Large dependencies bloat your bundle size, increasing download and parse times. Un-memoized expensive calculations in the render path cause slow re-renders.
2. **How:**
  1. Change `import _ from 'lodash'` to `import sortBy from 'lodash/sortBy'`. Use a bundle analyzer (like `rollup-plugin-visualizer` for Vite) to confirm the change.
  2. For the date formatting, create a single `Intl.DateTimeFormat` instance outside your component and reuse it. Or, wrap the calculation in `useMemo` within your component.
  3. Strategically apply `React.memo` to your `ArticleItem` component to prevent re-renders when its props haven't changed. Be prepared to document cases where memoization doesn't help or even adds overhead.

## 4. Optimize Image Delivery:

1. **Why:** Images are often the largest assets on a page. Proper optimization is key to a good LCP and CLS score.
2. **How:**
  1. Compress your hero image. Use a tool like Squoosh to convert it to a modern format like WebP and reduce its file size.
  2. Add explicit `width` and `height` attributes to the `<img>` tag to prevent layout shifts.
  3. Use the `srcset` attribute to provide different image sizes for different viewport widths.
  4. Add `loading="lazy"` to defer the loading of off-screen images (though for the main hero image, you might want it to load eagerly).

## 5. Implement Code Splitting:

1. **Why:** Reduces the size of the initial JavaScript payload, leading to a faster Time to Interactive (TTI).
2. **How:** If your app had multiple pages, you would use route-based code splitting. For a single-page app like this, you can still split out heavy

components that are not immediately needed. Use `React.lazy` and `Suspense`

## partnr

After each of these steps, re-run Lighthouse and the Performance profiler, and meticulously document the results in `PERFORMANCE.md`.

### 5. Final Deliverables & Documentation

Your submission will be judged not just on the final performance score but on the quality of your process and documentation.

1. **Code:** The final, optimized code should be on the `main` or `master` branch.
2. **Slow Version:** The initial, unoptimized code must be on a branch named `slow-version`.
3. **README.md:** A high-quality README with:
  1. A description of the project.
  2. Clear instructions on how to install dependencies and run the project locally (for both the slow and fast versions).
4. **PERFORMANCE.md:** The completed performance audit file containing:
  1. The initial baseline measurements table.
  2. A section for each optimization, detailing the change made and the before-and-after metrics. Explain *why* the metric improved.

### 6. FAQ

#### 1. Q: Why do I have to build the slow version first?

1. **A:** This is fundamental to performance work. Without a concrete "before" state, you cannot scientifically measure the impact of your "after" state. It forces you to understand the *cause* of a problem before you can apply the correct solution.

#### 2. Q: What if `React.memo` or `useMemo` makes performance worse?

1. **A:** This is an excellent observation! Memoization is not free; it adds overhead for checking props and managing cached values. If a component re-renders very frequently with different props anyway, the cost of memoization can be greater than the cost of re-rendering. Document this finding in your `PERFORMANCE.md` —it demonstrates a deep understanding of React's render lifecycle.

#### 3. Q: How can I reliably measure INP?

1. **A:** INP is a field metric, meaning it's best measured with real user data. For lab testing, the best correlate is Total Blocking Time (TBT) from a Lighthouse report. You can also use the Chrome DevTools Performance panel to identify long tasks that occur after user input (e.g., typing in a filter box). These long tasks are what cause poor INP.

## Core Requirements

### partnr

Compose a `docker-compose up` command should pass and start the application, making it accessible in a web browser.

#### Contract:

1. A `docker-compose.yml` file must be present in the repository root.
2. At least one `Dockerfile` must be present.
3. The `docker-compose up` command must successfully start a web server serving the application on a specified port.
4. The application service in `docker-compose.yml` must include a `healthcheck`.

#### Verification:

1. Check for the existence of `docker-compose.yml` and `Dockerfile`.
2. Run `docker-compose up -d --build`.
3. Verify that all services become healthy within 2 minutes.
4. Make an HTTP GET request to the specified port (e.g., `http://localhost:3000`) and expect a `200 OK` response with HTML content.

2. A Git branch named `slow-version` must exist in the repository. This branch must contain the initial, unoptimized version of the application, demonstrating the anti-patterns described in Phase 1.

#### Contract:

1. A branch named exactly `slow-version` must be present in the Git history.

#### Verification:

1. Run `git ls-remote --heads origin` OR `git branch -r`.
2. Verify that `origin/slow-version` exists in the output.

3. A detailed performance report file named `PERFORMANCE.md` must be present in the root of the repository.

#### Contract:

1. A file named exactly `PERFORMANCE.md` must exist at the project root.
2. The file must contain markdown content, including at least one table.

#### Verification:

1. Check for the existence of `PERFORMANCE.md` at the repository root.
2. Verify the file is not empty and contains the markdown syntax for a table ( `|` ).

4.



An `.env.example` file must be present in the repository root, documenting all

## partnr

1. A file named `.env.example` must exist at the project root.
2. It should contain key-value pairs for environment variables, such as `PORT`.

### Verification:

1. Check for the existence of `.env.example`.
2. Parse the file to ensure it contains at least one variable assignment (e.g., `PORT=3000`).

- 
5. The main hero image in the final optimized application must use modern optimization attributes.

### Contract:

1. The primary `<img>` element, identifiable with `data-testid="hero-image"`, must be present.
2. This element must have `width`, `height`, and `srcset` attributes.

### Verification:

1. Start the optimized application.
2. Use a browser automation tool (like Playwright) to navigate to the main page.
3. Locate the element with `data-testid="hero-image"`.
4. Assert that the element has non-empty `width`, `height`, and `srcset` attributes.

- 
6. The list of news articles must be rendered using virtualization to maintain a small DOM size.

### Contract:

1. The container for the article list should have a `data-testid="article-list"`.
2. The number of direct child elements within this container that represent articles (e.g., having `data-testid="article-item"`) must be significantly less than the total number of articles (500).

### Verification:

1. Start the optimized application.
2. Use a browser automation tool to navigate to the main page.
3. Wait for the article list to be populated.
4. Count the number of elements with `data-testid="article-item"` inside the `data-testid="article-list"` container.
5. Assert that the count is less than 50 (a reasonable buffer for a virtualized list).

7. The production build of the application must produce multiple JavaScript chunks,

partnr

1. The production build output directory (e.g., `build/static/js` or `dist/assets`) must exist after running the build script.
2. This directory must contain more than one `.js` file.

**Verification:**

1. Run the production build command specified in `package.json` (e.g., `npm run build`).
2. After the build completes, list the contents of the JavaScript output directory.
3. Verify that there are at least two distinct `.js` files (e.g., `main.[hash].js` and `1.[hash].js`).

- 
8. The final production bundle must not include the entire `lodash` library. Only the imported modules should be present.

**Contract:**

1. The source code must demonstrate cherry-picked imports (e.g., `import sortBy from 'lodash/sortBy'`).
2. A bundle analysis report file must be generated at build time, e.g., `stats.html`.

**Verification:**

1. Run the production build command.
2. Check for the existence of a `stats.html` or similar bundle analysis report.
3. (Alternative) Grep the final bundled JavaScript files for a string unique to an unimported lodash function (e.g., `isEqual`). The string should not be found.

[About Us](#)

[Contact Us](#)

[Privacy Policy](#)

[Terms and Conditions](#)



partnr